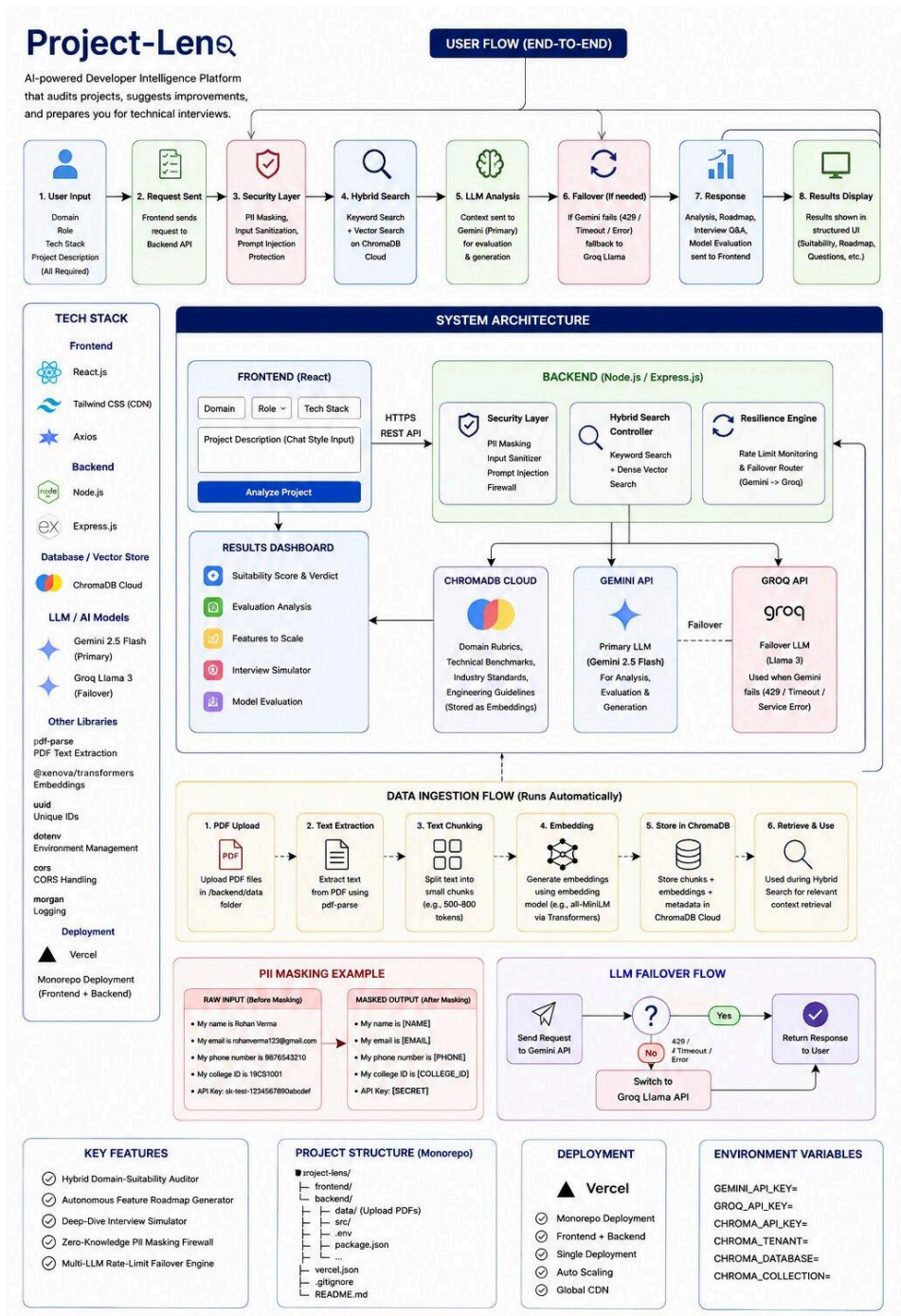


ProjectLens: System Architecture & Design Document



1. High-Level Architecture Overview

For ProjectLens, I decided to go with a decoupled Client-Server architecture utilizing a Retrieval-Augmented Generation (RAG) pipeline. I split the frontend and backend completely because I knew the LLM generation and vector retrieval steps could introduce high latency, and I needed the React client to remain responsive while the backend handled the heavy lifting.

The system operates as an API-driven orchestrator: the frontend handles state and UI, while the Node.js backend acts as a secure gateway that manages data sanitization, talks to the local vector database, and coordinates with the external Gemini LLM.

3. Step-by-Step Data Flow

(Reference the attached architecture diagram for the visual flow of these steps)

1. **Payload Submission:** The user submits their project details via the React UI. A POST request containing a JSON payload is sent to the Express backend.
2. **PII Interception:** The Express router passes the payload to the security middleware. The code scrubs the text of sensitive identifiers and passes the clean text forward.
3. **Semantic Search:** The backend takes the sanitized project description and queries the local ChromaDB instance. Chroma returns the top matched engineering standards for the user's specific domain (e.g., Fintech security requirements).
4. **Prompt Orchestration:** The Node.js server dynamically constructs a massive string. It stitches together the user's tech stack, the retrieved ChromaDB context, and the strict "Principal Engineer" system persona.
5. **LLM Execution:** The orchestrated prompt is fired off to the Gemini API via an asynchronous call.
6. **Response Parsing & Delivery:** Gemini returns a stringified JSON object. The backend parses it to ensure it didn't break structure, and then sends the 200 OK response back to the React client to populate the Suitability Score, Architectural Gaps, and Interview Simulator.